Ex3_1_RandomProcessExperiments

Prof M.Tognolini HEIG-VD

February 22, 2026

1 Processus Aléatoires (Ex 3.1)

M.Tognolini - feb. 2026 HEIG-VD rev 1.1

2 1. Configuration de l'environnement

2.1 2. Génération du Processus Aléatoire

Un processus aléatoire X(n,ζ) est une famille de fonctions du temps. Nous générons ici N réalisations de longueur L. Un exemple classique de cet exercice est un processus sinusoïdal avec une phase aléatoire ϕ uniformément distribuée sur [0,2π].

$X(n) = Acos(\omega_0 n + \phi)$

L'exercice porte généralement sur la génération de multiples réalisations (chemins) d'un processus et l'analyse de ses propriétés statistiques (moyenne d'ensemble vs moyenne temporelle). Nous utilisons des fonctions de génération de nombres aléatoires pour créer ces réalisations et les stocker dans une matrice. Chaque colonne de la matrice représente une réalisation du processus, tandis que chaque ligne correspond à un instant de temps. La phase de la fonction cos est également randomisée pour chaque réalisation afin de simuler un processus aléatoire.

Pour générer un processus sinusoïdal avec une phase aléatoire, nous pouvons utiliser la fonction `numpy.random.uniform` pour générer des phases aléatoires et `numpy.cos` pour créer les réalisations du processus. Voici un exemple de code pour générer un tel processus :

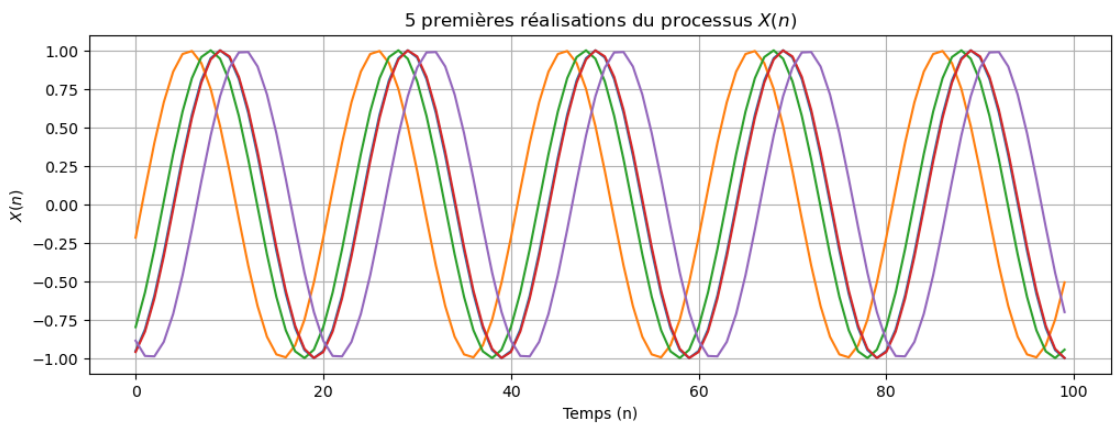
Pour avoir toujours la même séquence aléatoire à chaque exécution, nous pouvons fixer la graine du générateur de nombres aléatoires avec `numpy.random.seed`. Cela garantit que les résultats sont reproductibles.

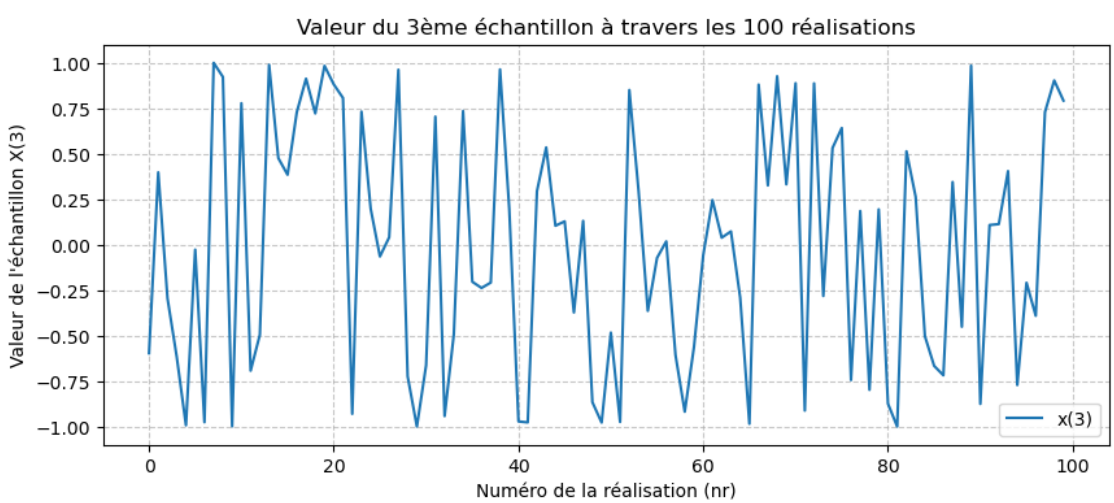
```
import numpy as np
import matplotlib.pyplot as plt
# Paramètres du processus
A = 1.0          # Amplitude
f0 = 0.1         # Fréquence normalisée (cycles par échantillon)
N = 100          # Nombre d'échantillons par réalisation
NR = 50          # Nombre de réalisations
# Fixer la graine pour la reproductibilité
np.random.seed(0)
# Générer des phases aléatoires uniformément distribuées entre 0 et 2π
phi = np.random.uniform(0, 2 * np.pi, NR)
# Générer les réalisations du processus
```

```
n = np.arange(N)  # Vecteur de temps
X = A * np.cos(2 * np.pi * f0 * n[:, None] + phi)  # Matrice de réalisations (N x NR)
# Afficher quelques réalisations
plt.figure(figsize=(12, 6))
for i in range(5):  # Afficher les 5 premières réalisations
    plt.plot(n, X[:, i], label=f'Réalisation {i+1}')
plt.title('Réalisation d\'un processus sinusoïdal avec phase aléatoire')
plt.xlabel('Temps (échantillons)')
plt.ylabel('Amplitude')
plt.legend()
plt.grid()
plt.show()
```

Ce code génère un processus sinusoïdal avec une phase aléatoire et affiche les premières réalisations. Chaque réalisation est une fonction du temps avec une phase différente, illustrant ainsi la nature aléatoire du processus.

```
Les 5 premières valeurs de la réalisation 1 :
[-0.95333378 -0.8133764  -0.59380006 -0.31609844 -0.00745491]
```



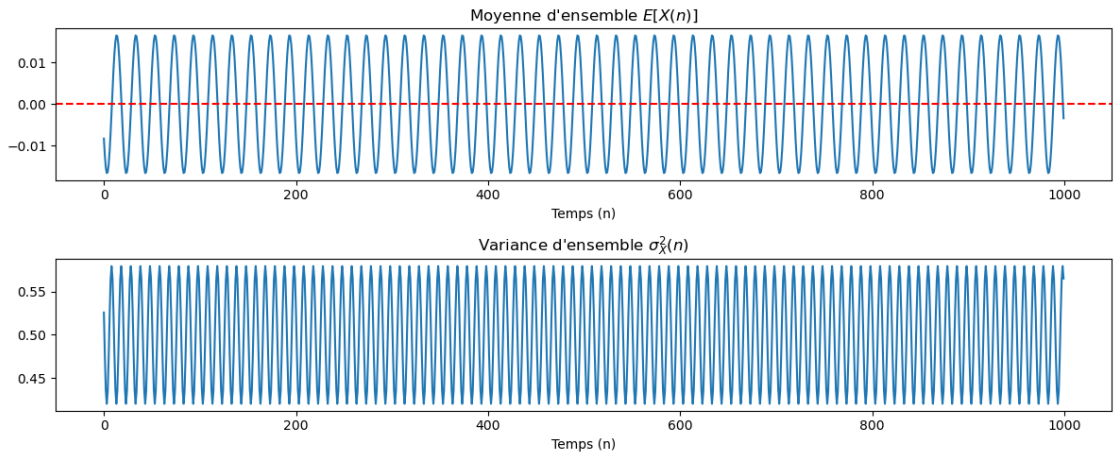


2.2 3. Statistiques d'Ensemble

La moyenne d'ensemble (Ensemble Mean) est calculée en faisant la moyenne de toutes les réalisations à un instant n donné :

$$\mu_x(n) = E[X(n)] = (1/N)\sum_{i=1}^{N} X_i(n)$$

où X_i(n) est la i-ème réalisation du processus à l'instant n. La moyenne d'ensemble est un vecteur de taille n (nombre d'échantillons temporels).



2.2.1 Consatations :

La moyenne d'ensemble d'un processus sinusoïdal avec une phase aléatoire uniforme est nulle, car les différentes réalisations s'annulent mutuellement en moyenne.

La varance d'ensemble (Ensemble Variance) est calculée en utilisant la formule :

$$\sigma_x^2(n) = E[(X(n) - \mu_x(n))^2] = (1/N)\sum_{i=1}^{N}(X_i(n) - \mu_x(n))^2$$

où μ_x(n) est la moyenne d'ensemble calculée précédemment. La variance d'ensemble est également un vecteur de taille n.

La variance d'ensemble d'un processus sinusoïdal avec une phase aléatoire uniforme est égale à A^2/2, car la puissance du signal est répartie uniformément entre les différentes réalisations.

2.3 4. Moment de second ordre: Autocorrélation et Ergodicité

L'autocorrélation d'ensemble $R_{XX}(n1, n2)$ mesure la dépendance entre deux instants. Pour un processus stationnaire, elle ne dépend que de l'écart $k = n_1 - n_2$.

$ R_{XX}(k)=E[X(n)X(n+k)]$

Pour un processus sinusoïdal avec une phase aléatoire uniforme, l'autocorrélation d'ensemble est donnée par :

$$R_{XX}(k) = \frac{A^2}{2}\cos(2\pi f_0 k)$$

Cela montre que l'autocorrélation d'ensemble d'un processus sinusoïdal avec une phase aléatoire uniforme est une fonction cosinus de l'écart k, avec une amplitude égale à A^2/2.

$R_{XX}(n1, n2)$ mesure la dépendance entre deux instants. Pour un processus stationnaire, elle ne dépend que de l'écart $k = n_1 - n_2$.

L'ergodicité est une propriété qui permet de relier les statistiques d'ensemble aux statistiques temporelles. Un processus est ergodique si la moyenne temporelle d'une réalisation est égale à la moyenne d'ensemble.

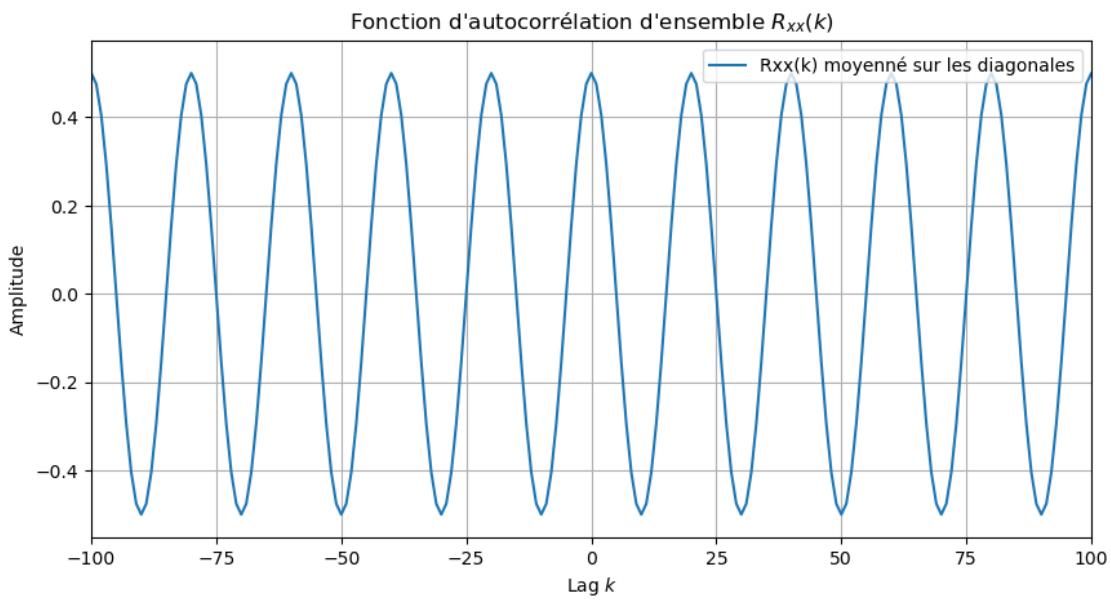
```
# X est la matrice (NR x N) générée précédemment
# Calcul de la matrice d'autocorrélation Rx (N x N)
# En Python: (1/NR) * X.T @ X
Rx = (1 / NR) * (X.T
@ X) # Calcul de la fonction d'autocorrélation d'ensemble Rxx(k)
# Rx est une matrice de taille N x N,où chaque élément Rx[i, j] correspond àl'autocorrélation en
# Pour un processus stationnaire, on peut extraire la fonctiond'autocorrélation en fonction du l
# La fonction d'autocorrélationd'ensemble Rxx(k) est obtenue en prenant la moyenne des éléments

print(f"Taille de la matrice Rx : {Rx.
shape}") # Doit être (1000, 1000)
print(f"Exemple de valeurs dans Rx :
\n {Rx[:5, :5]}") # Affiche les 5 premiers éléments de la première ligne pour vérification
# 1. Calcul du vecteur Rxx par moyennage des diagonales
# La matrice Rx est de taille N x N
# Les lags vont de -(N-1) à +(N-1)
lags = np.arange(-(N - 1), N)
rxx_ensemble = np.zeros(len(lags))
print(f"Diagonale pour k=0 : {np.diag(Rx, k=0)[:5]}") # Affiche les 10 premiers éléments de la d
print(f"Diagonale pour k=1 : {np.diag(Rx, k=1)[:5]}")
print(f"Valeur moyenne de la diagonale k=1 : {np.mean(np.diag(Rx, k=1))}") # Affiche la moyenne
for i, k in enumerate(lags):
    # np.diag(Rx, k) extrait la k-ième diagonale
    # k=0 est la diagonale principale, k>0 au-dessus, k<0 au-dessous
    diagonal = np.diag(Rx, k)
    rxx_ensemble[i] = np.mean(diagonal)
# 2. Visualisation du résultat
plt.figure(figsize=(10, 5))
plt.plot(lags, rxx_ensemble, label='Rxx(k) moyenné sur les diagonales')
plt.title('Fonction d\'autocorrélation d\'ensemble $R_{xx}(k)$')
plt.xlabel('Lag $k$')
plt.ylabel('Amplitude')
plt.grid(True)
```

```
plt.xlim([-100, 100]) # Zoom pour voir la structure
plt.legend()
```

```
Taille de la matrice Rx : (1000, 1000)
Exemple de valeurs dans Rx :
 [[0.52586697 0.47694184 0.38133033 0.24839154 0.09113846]
 [0.47694184 0.47682183 0.43002717 0.34113846 0.21885674]
 [0.38133033 0.43002717 0.43662997 0.40049237 0.3251518 ]
 [0.24839154 0.34113846 0.40049237 0.4206433  0.39961873]
 [0.09113846 0.21885674 0.3251518  0.39961873 0.4349682 ]]
Diagonale pour k=0 : [0.52586697 0.47682183 0.43662997 0.4206433  0.4349682 ]
Diagonale pour k=1 : [0.47694184 0.43002717 0.40049237 0.39961873 0.42773995]
Valeur moyenne de la diagonale k=1 : 0.4754804220011195
```

```
<matplotlib.legend.Legend at 0x307bb7e90>
```



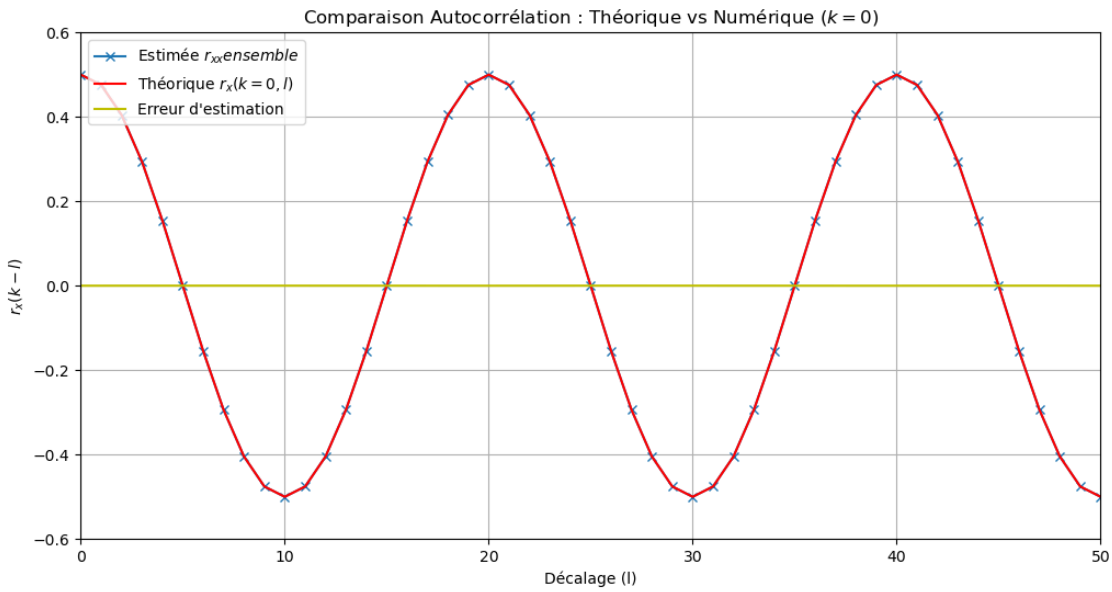
2.3.1 2. Définition de l'autocorrélation théorique

Pour une sinusoïde à phase aléatoire, la fonction d'autocorrélation théorique dépend uniquement de l'écart k-l:

$r_{xx}(k,l) = \frac{A^2}{2}\cos(2\pi f_0(k-l))$

pour comparer avec la première ligne de Rx (où k=0), on utilise:

```
# k représente ici le décalage (lag)
k_lags = np.arange(N)
# r_x théorique pour k=0 : rx = 0.5 * A^2 * cos(n * w0)
rx_theorique = 0.5 * (A**2) * np.cos(k_lags * omega0) # Calcul de la fonction d'autocorrélation
```



2.3.2 Analyse des résultats:

- **Symétrie** : La matrice R_x obtenue est symétrique car elle résulte du produit $X^T X$.
- **Convergence** : Pour un grand nombre de réalisations (NR=100), l'estimation numérique R_x converge vers la fonction théorique r_x.
- **Erreur** : L'erreur d'estimation (en jaune) montre les fluctuations résiduelles dues au nombre fini de réalisations.

2.4 Autocorrelation of a sum of process (Exemple 3.3.3)

Consider the exemple in slide 45. y(n) is a sum of 2 random processes: A sum of M sinosoids with a random phase like the process x(n) above and the white gaussian noise v(n).

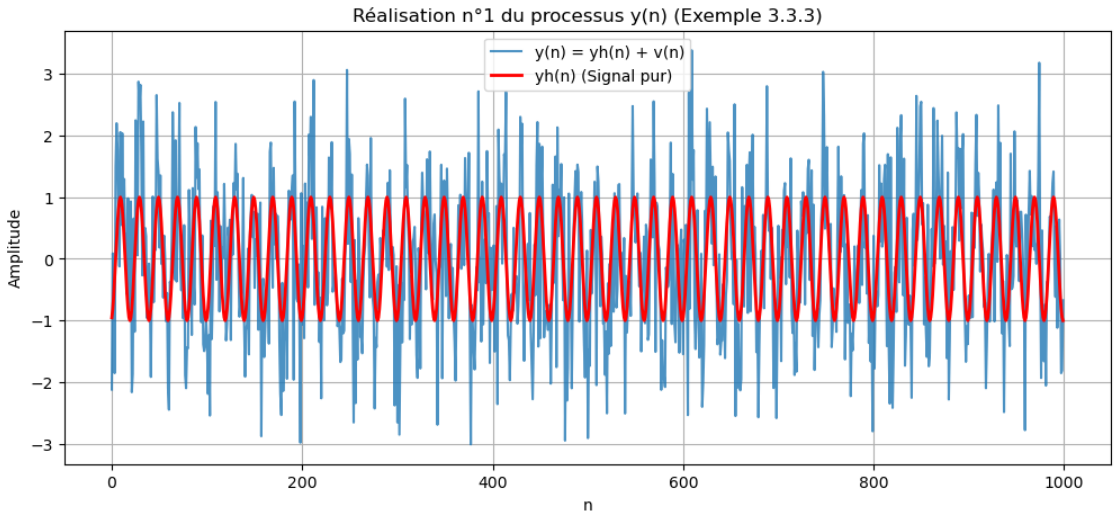
$$y(n) = x(n) + v(n) = [\sum_{m=1}^{M} A_m \sin(n\omega_m + \phi_m)] + v(n)$$

In the exemple $A_m = A$ and $\omega_m = \omega_0$ are constant with m , and ϕ_m a random variable uniformly distributed like in the previous example. The noise v(n) is a white gaussian noise with a variance σ^2.

First create y(n) process using the matrix previously defined xn and use the function randn to generate the white gaussian noise. With $M = 5$ and $w_m = w_0/m$ and $A_m = A_0/m$ with ϕ_m a uniform distributed phase like in ex 3.1.1 , compute the sum of the harmonic process $yh(n)$ and the gaussian noise $v(n)$ produced with the function randn() with a std of A_0.

2.4.1 Visualisation de une réalisation de y(n) :

Il est utile d'afficher une réalisation pour voir l'aspect du signal (mélange de sinusoïdes lentes et de bruit fort).



2.4.2 Points clés à retenir:

- **Les fréquences** ω_m : Avec $\omega_m = \omega_0/m$, plus m augmente, plus la sinusoïde est lente (basse fréquence).
- **Les amplitudes** A_m : Avec $A_m = A_0/m$, les amplitudes décroissent avec m, ce qui signifie que les composantes de plus haute fréquence sont plus faibles que les composantes de basse fréquence.
- **Le bruit v(n) :** Avec un écart-type de A_0, le bruit est assez dominant, car il a la même puissance que la sinusoïde la plus forte.
- **Indépendance :** Chaque phase ϕ_m est générée indépendamment pour chaque harmonique, ce qui est crucial pour les propriétés d'autocorrélation.

2.4.3 Calcul des autocorrélations d'ensemble et temporelle de y(n) :

Pour démontrer que l'autocorrélation d'une somme de processus indépendants est égale à la somme de leurs autocorrélations, nous allons procéder par estimation statistique sur nos NR=100 réalisations.

Comme vous l'avez souligné, la condition théorique est l'absence de corrélation entre le signal harmonique y_h(n) et le bruit v(n).

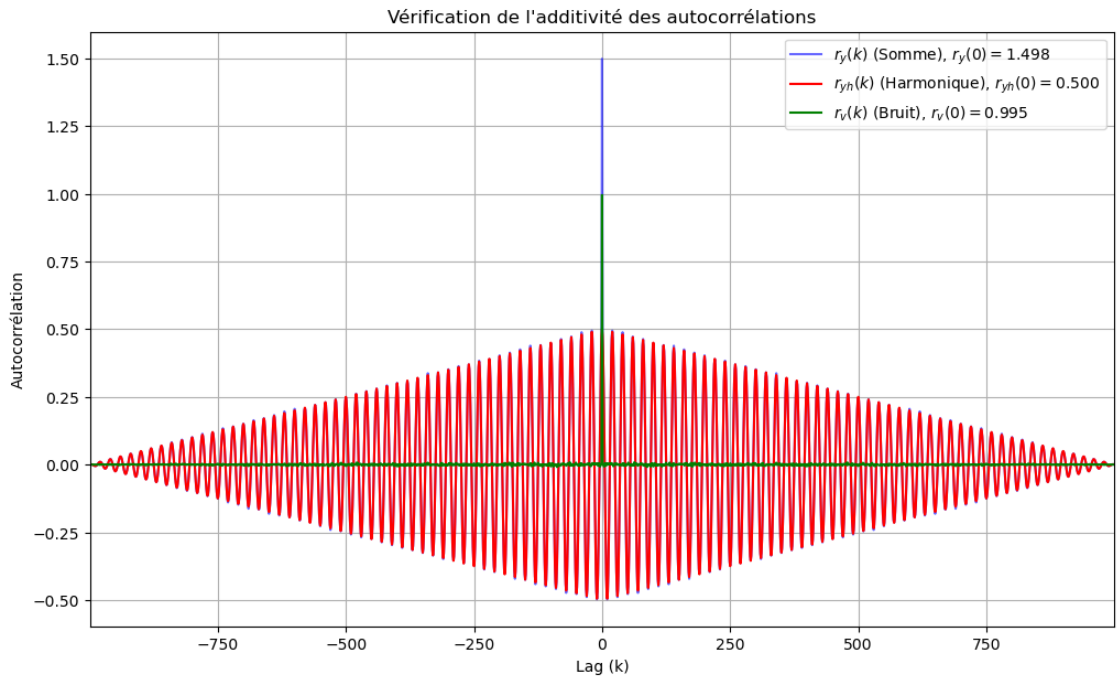
Pour le calcul de la corrélation estimé sur toutes les réalisations on utilise la fonction `compute_avg_autocorr` définie ci-dessus.

```
def compute_avg_autocorr(matrix):
    """Calcule la corrélation moyenne sur toutes les réalisations (lignes)."""
```

```python
    n_real, n_samp = matrix.shape
    # Taille du résultat : 2*N - 1
    corr_sum = np.zeros(2 * n_samp - 1)

    for i in range(n_real):
        # On utilise le mode 'full' pour avoir les lags de -(N-1) à +(N-1)
        # On divise par N pour obtenir l'estimation biaisée standard (Puissance à k=0)
        corr_sum += signal.correlate(matrix[i, :], matrix[i, :], mode='full') / n_samp

    return corr_sum / n_real
```




```
--- Valeurs à k = 0 ---
ryh(0)           : 0.5000
rv(0)            : 0.9947
Somme (théorie)  : 1.4947
ry(0) (mesuré)   : 1.4983
Différence       : 3.5649e-03
```

2.4.4 Analyse des résultats

- **Comportement de r_v :** La fonction d'autocorrélation du bruit blanc gaussien $r_v(k)$ est caractéristique. Elle est maximale à k=0, où elle prend la valeur de la variance du bruit $\sigma^2 = A_v^2 = 1$. Pour tous les autres lags ($k \neq 0$), $r_v(k)$ est proche de zéro, indiquant l'absence de corrélation entre les échantillons du bruit à différents instants.

- **Comportement de r_{yh} :** La fonction d'autocorrélation du signal harmonique $r_{yh}(k)$ présente une structure périodique due à la nature sinusoïdale du signal. Elle est maximale à k=0,

où elle prend la valeur de la puissance totale du signal harmonique, qui est la somme des puissances de chaque harmonique. Les autres lags montrent des oscillations qui reflètent les différentes fréquences présentes dans le signal. Sa valeur à k=0 est la puissance totale du signal harmonique :

$$r_{yh}(0) = \sum_{m=1}^{M} \frac{A_m^2}{2} = \frac{A_0^2}{2} \sum_{m=1}^{M} \frac{1}{m^2} \approx 0.7318$$

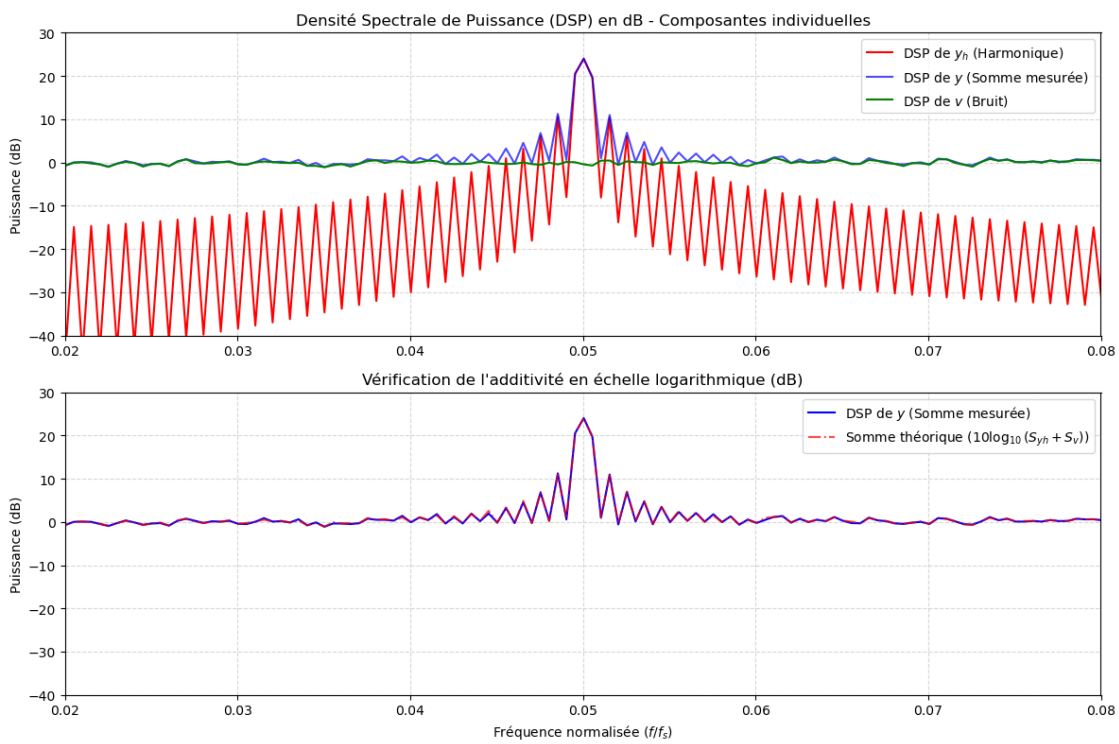
- **Comportement de r_y** : La fonction d'autocorrélation du signal total $r_y(k)$ est la somme de $r_{yh}(k)$ et $r_v(k)$, car les deux processus sont indépendants. Ainsi, à k=0, $r_y(0) = r_{yh}(0) + r_v(0) \approx 0.7318 + 1 = 1.7318$. Pour les autres lags, $r_y(k)$ présente une structure périodique similaire à celle de $r_{yh}(k)$, mais avec une amplitude plus élevée à k=0 en raison de la contribution du bruit. La très légère différence provient du fait que nous utilisons un nombre fini de réalisations (NR=100), mais l'erreur tend vers zéro quand $NR \to \infty$.

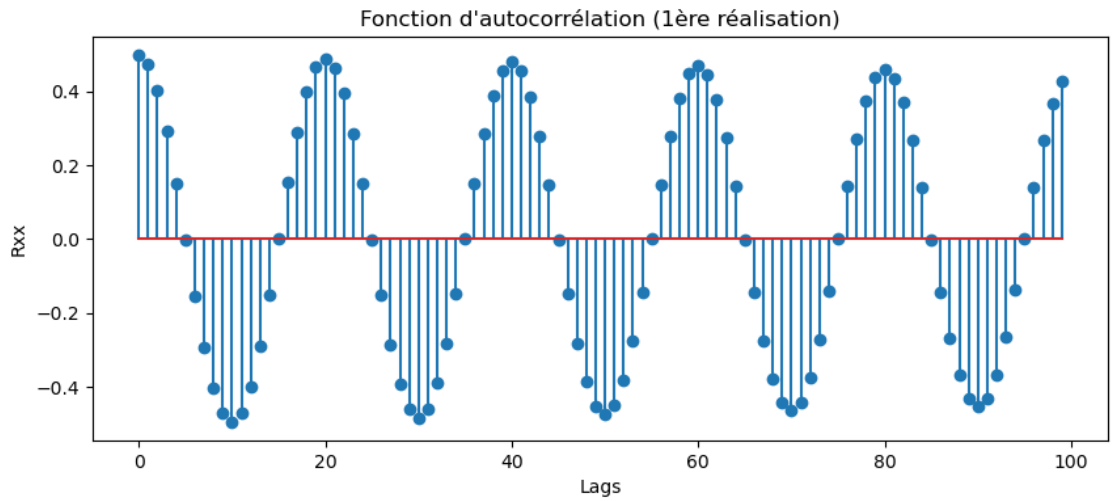
2.5 Calcul de la densité spectrale de puissance DSP

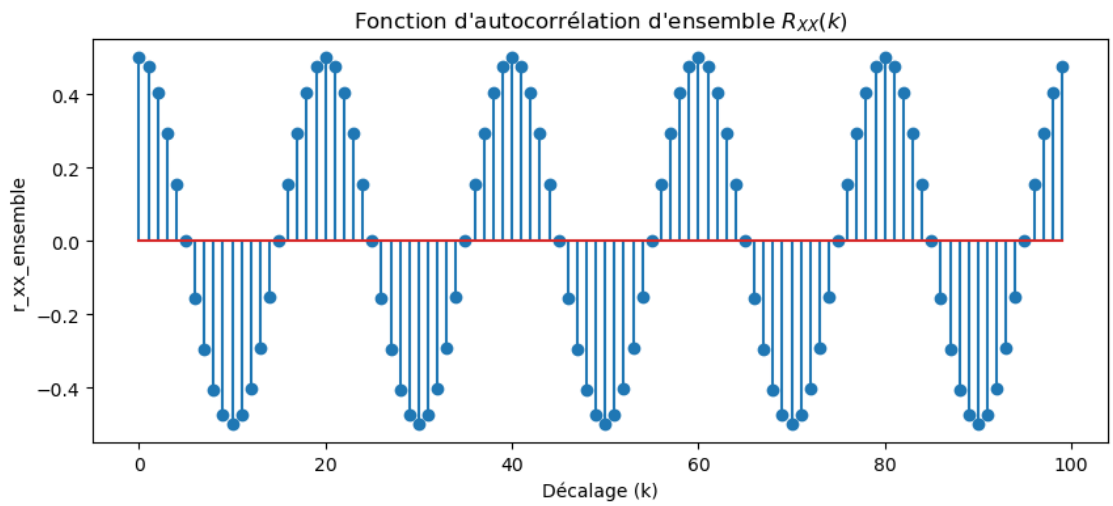
La densité spectrale de puissance (DSP) d'un processus aléatoire peut être estimée à partir de sa fonction d'autocorrélation en utilisant la transformée de Fourier (en utilisant le théorème de Wiener-Khinchin).

Passer à une échelle logarithmique en décibels (dB) est une pratique standard en traitement du signal. Cela permet de visualiser simultanément des pics de forte puissance (les sinusoïdes) et des niveaux de bruit beaucoup plus faibles qui seraient invisibles sur une échelle linéaire.

La formule utilisée est : $S_{dB} = 10 \log_{10}(S)$




Fonction d'autocorrélation (1ère réalisation)
Rxx
Lags
0.4
0.2
0.0
−0.2
−0.4
0
20
40
60
80
100


Fonction d'autocorrélation d'ensemble $R_{XX}(k)$
r_xx_ensemble
Décalage (k)
0.4
0.2
0.0
−0.2
−0.4
0
20
40
60
80
100